ML-Based Fake Website and Online Fraud Detection System

Project Report: Features, Future Enhancements, Execution Guide, and Sample Input/Output

1. Introduction

This project is a web-based security platform that detects phishing / fake websites using a Machine Learning model and evaluates online payment transactions for fraud using a rule-based engine. Every scan and transaction check is stored in a database, so the system also provides history, alerts, a dashboard and downloadable reports.

Layer	Technology
Backend	Python 3.10+ (3.12 recommended), Django 5, Django REST Framework
Frontend (primary)	Django server-rendered HTML templates (no separate frontend server needed)
Frontend (legacy, optional)	React + Vite + TanStack (folder legacy_react_frontend/)
Machine Learning	Scikit-Learn Random Forest (180 trees, max depth 13), joblib model file model.pkl
Database	SQLite (db.sqlite3, auto-created, zero setup)
Network lookups	python-whois, socket (DNS), requests (redirects)
Training dataset	PhiUSIIL Phishing URL Dataset (235,000+ labeled URLs)

2. Existing Features

2.1 Website (URL) Scanner - ML based

- The user enters a URL and the system extracts **14 lexical / structural features**.
- A trained **Random Forest** gives a phishing probability, converted to a **risk score from 0 to 100**.
- Classes: **Legitimate** (below 40), **Suspicious** (40 to 74), **Fraudulent** (75 and above).
- If model.pkl is missing, a heuristic rule engine is used automatically, so the app never crashes.
- Human-readable **reasons** are generated (no HTTPS, many hyphens, risky TLD, shortener...), making the result explainable.

#	Feature	Meaning
1	https	URL uses HTTPS
2	domain_length	Length of domain name
3	url_length	Total URL length
4	hyphens_in_domain	Number of '-' in domain
5	digits_in_host	Digits in host name
6	has_ip_address	Raw IP used instead of domain
7	has_at_symbol	'@' present in URL
8	subdomain_count	Nested subdomains (ignores www and ccTLDs like .co.in)
9	suspicious_keywords_in_host	login, verify, secure, bank, paypal etc. inside domain
10	suspicious_keywords	Same keywords anywhere in URL
11	suspicious_tld	Risky TLDs: .tk .xyz .top .click .zip

#	Feature	Meaning
12	is_shortener	bit.ly, tinyurl.com, t.co and similar
13	has_query	Query string present
14	special_chars	Count of @ - _ % = & ? #

2.2 Domain Intelligence (offline-safe)

- **WHOIS domain age**, **DNS resolution** and **HTTP redirect count** are fetched with short timeouts (1 to 1.5 s).
- If offline or a lookup fails, the value is stored as *unknown* and **no penalty** is applied.
- Domains younger than 30 days raise the risk; domains older than one year lower it.

2.3 Online Transaction Fraud Checker - Rule based

Rule-based on purpose, because real banking datasets are not public. The UI and database label it clearly.

Rule	Condition	Points
High amount	Amount 1,500+ / 5,000+	+20 / +30
Country mismatch	Card country differs from billing country	+25
High-risk merchant	Crypto, gambling, casino, forex, gift cards, luxury goods, wire transfer	+25
New device	Unverified / new device	+15
First-time merchant	No earlier purchase with merchant	+10
Irreversible payment	Crypto, wire transfer, prepaid card	+15

Risk level: **Low** (below 35), **Medium** (35 to 69), **High** (70 and above), each with a recommended action (OTP, 2FA step-up, manual review).

2.4 Alerts and Safe-Browsing Tips

- Automatic **High** (critical) and **Medium** (warning) alerts for risky sites; a separate alert for raw IP address usage.
- Context-aware tips, e.g. avoid entering data on non-HTTPS sites.

2.5 Web Pages

Page	URL	Purpose
Home	/	System overview, KPIs, sample links
Scan Website	/scan/	Enter URL, risk meter, features, reasons
Check Transaction	/transaction/	Transaction fraud form
Dashboard	/dashboard/	Totals, class distribution, recent items
History	/history/	Audit log with search and class filter (latest 150)
Alerts	/alerts/	Alert centre filtered by severity
Reports	/reports/	Summary analytics, CSV export, printable report

2.6 Reports, REST API, Database, Testing

- **CSV export**: /reports/export-csv/ **Printable report**: /reports/printable/
- **REST API**: GET /api/health/ and POST /api/scan/ (result also saved to database).

- **Models**: ScanRecord, Alert, TransactionCheck.
- **Pipeline modules**: data collection, preprocessing, feature extraction, model building (Random Forest vs Logistic Regression), alerts, reports; train_model.py retrains the model.
- **test_all_modules.py** has 9 test stages; **all 9 passed** when run for this report.

3. Features That Can Be Added

3.1 Better Detection

1. **Page-content analysis**: password fields, login form posting to another domain, hidden iframes, external-link ratio.
2. **SSL certificate check**: issuer, validity, expiry, self-signed.
3. **Typosquatting / brand impersonation** using Levenshtein distance (paypa1.com, g00gle.com).
4. **Homograph / Punycode** (xn--) detection.
5. **More URL features**: entropy, path depth, port, digit ratio, .exe / .apk links.
6. **Threat-intel APIs**: Google Safe Browsing, VirusTotal, PhishTank, OpenPhish.
7. **Whitelist / blacklist** management.

3.2 Better Machine Learning

1. Compare **XGBoost / LightGBM / Gradient Boosting** with Random Forest.
2. **SHAP / feature importance** so explanations come from the model itself.
3. **Model Performance page**: accuracy, precision, recall, F1, confusion matrix.
4. Cross-validation and probability calibration.
5. Scheduled **retraining** with user-confirmed scans (feedback loop).

3.3 Transaction Fraud

1. ML model on Kaggle **Credit Card Fraud** or **IEEE-CIS** dataset.
2. Velocity features (transactions per hour/day, spend vs average).
3. **POST /api/transaction/** endpoint.
4. Time-of-day, IP geolocation and merchant-reputation rules.

3.4 Platform and UI

1. **User login and roles** (Admin / User) with per-user history.
2. **Charts** on dashboard (Chart.js).
3. **Bulk scan** from CSV upload.
4. **PDF and Excel** report export.
5. **Email / SMS / Telegram alerts** for critical detections.
6. **Chrome extension** that warns before a risky page loads.
7. **Multi-language UI** (Tamil / Hindi / English) and dark mode.

8. **Swagger / OpenAPI** docs and API-key authentication.

3.5 Engineering and Security Hardening

1. SECRET_KEY from environment, DEBUG=False, restricted ALLOWED_HOSTS before deployment.
2. Run WHOIS / DNS in background (Celery) and cache results; they currently run inside the request.
3. API rate limiting.
4. **Docker** and docker-compose; deploy to Render / Railway / AWS.
5. Remove unused PyMySQL dependency or add real MySQL / PostgreSQL option via environment variables.
6. Add dataset download link or script so retraining works out of the box.

4. How to Run the Project

4.1 Prerequisites

- Python 3.10+ (3.12 recommended) with pip.
- Node.js 18+ only for the optional legacy React frontend.
- No database installation needed (SQLite is built in).

4.2 Backend + Main Frontend (one server)

The main frontend is the Django template UI, so **one command starts both backend and frontend**.

Windows (easiest)

```
start-backend.bat
```

Linux / macOS

```
chmod +x start-backend.sh
./start-backend.sh
```

Manual steps

```
cd backend
python -m venv venv
venv\Scripts\activate          (Windows)
source venv/bin/activate       (Linux / macOS)
pip install -r requirements.txt
python manage.py makemigrations
python manage.py migrate
python manage.py runserver 8000
```

Open **http://localhost:8000** in the browser.

4.3 Legacy React Frontend (optional; backend must be running)

```
cd legacy_react_frontend
copy .env.example .env         (Windows)  |  cp .env.example .env  (Linux / macOS)
npm install
npm run dev
```

The .env file contains VITE_API_BASE_URL=http://localhost:8000. Vite prints the local address in the terminal; open that one.

4.4 Tests and Retraining

```
cd backend
python test_all_modules.py
```

To retrain (optional), place PhiUSIIL_Phishing_URL_Dataset.csv inside backend/ and run `python train_model.py`. The existing model.pkl is already trained.

4.5 Common Problems

Problem	Solution
python not found	Install Python with 'Add to PATH', or use py / python3
Port 8000 already used	python manage.py runserver 8001
Scikit-learn version warning on loading model.pkl	Install the training version of scikit-learn or retrain the model
WHOIS / DNS shows unknown	Normal when offline; no penalty is applied
React page cannot reach backend	Start the backend and check VITE_API_BASE_URL in .env

5. Sample Inputs and Outputs

All outputs were produced by actually running the project (without internet, so domain lookups show 'unknown').

5.1 Website Scan - API

Input

```
curl -X POST http://localhost:8000/api/scan/ -H "Content-Type: application/json" \
  -d '{"url":"http://secure-login-paypal-verify.xyz/update?id=123"}'
```

Output (shortened)

```
{
  "url": "http://secure-login-paypal-verify.xyz/update?id=123",
  "prediction": "Fraudulent",
  "risk_score": 98,
  "features": { "https": false, "domain_length": 30, "url_length": 51,
    "hyphens_in_domain": 3, "has_ip_address": false,
    "suspicious_keywords_in_host": 4, "suspicious_tld": true,
    "is_shortener": false, "has_query": true, "special_chars": 5 },
  "reasons": [
    "Unencrypted connection (No HTTPS)",
    "Multiple hyphens in domain name (3)",
    "Brand impersonation or security keyword embedded in domain",
    "High-abuse / risky Top-Level Domain (TLD)",
    "High count of URL special/encoded characters" ],
  "alerts": [ { "severity": "high",
    "title": "Critical Phishing Alert: Fraudulent Domain Detected" } ]
}
```

5.2 More Website Samples

Input URL	Prediction	Risk	Alert
https://www.google.com	Legitimate	1	None
https://github.com/anthropics	Legitimate	7	None
http://bit.ly/free-gift	Fraudulent	99	Critical alert
http://192.168.1.10/bank/login@evil.com	Fraudulent	98	Critical + Raw IP warning
http://secure-login-paypal-verify.xyz/update?id=123	Fraudulent	98	Critical alert

5.3 Error Responses

Input	Response
{} (no url)	{"error": "url is required"}
{"url": "google.com"}	{"error": "url must start with http:// or https://"}

On the /scan/ web page, a missing http:// is added automatically.

5.4 Transaction Check Samples (form at /transaction/)

Input	Output
Amount 120, Credit Card, Amazon / E-Commerce, card IN, billing IN, known merchant and device	Risk 0, Low Risk, no rules triggered. Transaction within safe thresholds.
Amount 6200, Cryptocurrency, CoinXYZ / Crypto Exchange, card IN, billing US, first-time merchant, new device	Risk 100, High Risk. Triggered: high value, country mismatch, high-risk category, new device, first-time merchant, irreversible payment. Action: 2FA step-up or manual review.

5.5 CSV Export Sample

```
Scan ID,Website URL,Classification,Risk Score (%),Scan Timestamp (UTC),Key Threat Indicators
9,http://secure-login-paypal-verify.xyz/update?id=123,Fraudulent,98,2026-11-11 15:45:13,Unencrypted connection
```

6. Verification Summary

Check	Result
Database migrations	Applied successfully
/api/health/	{"status": "ok"}
9 web pages (Home, Scan, Transaction, Dashboard, History, Alerts, Reports, CSV, Printable)	HTTP 200
test_all_modules.py	9 of 9 stages passed
ML model	Random Forest, 180 trees, max depth 13, loaded from model.pkl

7. Limitations

- Only URL-based features are used (page content is not analysed), so a well-crafted phishing URL on a normal-looking domain may be missed.
- Displayed reasons come from the heuristic engine while the score comes from the Random Forest, so they can occasionally differ.
- Transaction fraud is rule-based, not ML, because no public banking dataset is used.
- Domain lookups need internet; offline they are skipped without penalty.
- College prototype: debug mode on and no user login, so not hardened for production.

8. Conclusion

The project combines an ML-based phishing URL detector, offline-safe domain verification, a rule-based transaction fraud engine, automated alerts, persistent audit history and exportable reports in a single Django application. The enhancements in Section 3, especially page-content analysis, threat-intelligence APIs, SHAP explainability, an ML-based transaction model and user authentication, would make it a strong production-style system.